The Reggae Shark Protocol (v1.1.2): Formal Typestate Proofs, 4D Soft-Wall ODE Governance, LRSC Spectral Compression, and Merkle-DAG Provenance

Prince Upadhyay
Independent Researcher · ORCID: 0009-0002-1170-4487 · starxshark@gmail.com
Repository: github.com/reggaesharkk/The-Reggae-Shark-Protocol
Canonical Zenodo DOI: 10.5281/zenodo.23620074 (Supersedes v1.0.3 DOI: 10.5281/zenodo.23618033)
Licensing: MIT License (Code & Lean 4 Proofs) · CC BY 4.0 (Specification Manuscript)

October 24, 2026

Abstract

The Reggae Shark Protocol (v1.1.2) is a verifiable execution kernel and multi-cloud AI gateway that binds agent prompt dispatches to four mathematical and cryptographic verification layers: (1) an append-only, SHA-256 hash-chained **IntentSeal Verifiable Research Kernel (VRK v0.3.1)** ledger enforcing the `intentseal-vrk-claim-v2` schema with a commutative Merkle-DAG join-semilattice extension for distributed split-brain reconciliation; (1B) a machine-checked **Request-Lifecycle Typestate and Worker-Epoch Isolation** state machine (`TypestateLifecycle.lean`, 8 zero-`sorry` theorems) that rules out partial-initialization payload dereferencing and cross-epoch worker-slot reads within the modelled state machine; (2) a **4D Soft-Wall ODE Cognitive Dynamics Governor** $(\gamma, H_{\text{obs}}, \dot{I}, \Psi)$ backed by 17 Lean 4 pointwise boundary theorems (`ExistenceVerification.lean`), 9 3D soft-wall theorems (`BoundaryVerification.lean`), and an unconditionally box-invariant Patankar–Euler interval step; and (3) a **Low-Rank Spectral Context Memory (LRSC)** operator ($M = 99, K = 11$) with an orthogonal complement ($S_{11}^{\perp}$) anomaly detector.

1 Four-Layer Verification Architecture & Provenance

Every request submitted to `MultiProviderGateway.process_request` or the local loopback HTTP proxy (`POST http://127.0.0.1:8000/v1/execute`) traverses four deterministic verification gates:

Layer	Module / Artifact	Enforced Mathematical & Cryptographic Invariant
Layer 1	`shark_protocol/kernel.py`	**VRK v0.3.1 Hash Chain & Merkle-DAG Join**: Append-only SHA-256 chain anchored at trust root `f010a99d...05a0d0f` (10.5281/zenodo.23018459), with commutative `parent_hashes` reconciliation and mandatory `scope.applies_to` / `scope.does_not_imply` firewalls.
Layer 1B	`lean/TypestateLifecycle.lean`	**Typestate & Worker-Epoch Isolation**: 6-state request automaton (`UNINIT` → `PARTIAL_INIT` → `SEALED_ACTIVE` → `SHUTDOWN_CLEAN` → `ZEROED_FREED`) with `OOM_ABORT` → `ABORTED_POISONED`, `slot_epoch == worker_epoch` isolation, and thread-safe `RLock` fence sequencing.
Layer 2	`lean/ExistenceVerification.lean` `lean/BoundaryVerification.lean`	**4D Soft-Wall ODE Governor ($\mathcal{B}_4$)**: Adaptive RK4 and unconditionally box-invariant Patankar–Euler interval integration of $(\gamma, H_{\text{obs}}, \dot{I}, \Psi)$ inside $\mathcal{B}_4 = [0,10] \times [0,100] \times [0, \dot{I}_{\max,4D}] \times [0,1]$, backed by 26 zero-`sorry` Lean 4 boundary theorems.
Layer 3	`LRSCCompressor` (`kernel.py`)	**LRSC Odd-Ring Spectral Projection ($11/99 = 1/9$)**: Projects prompt context onto the $M = 99$ near-Nyquist cyclic basis, retains the $K = 11$ witness subset $S_{11} = \{0, 1, 3, 42..49\}$ (10.17605/OSF.IO/NM5BW), and monitors orthogonal $S_{11}^{\perp}$ energy.
Layer 4	`shark_protocol/gateway.py`	**Fail-Closed Multi-Cloud Gateway**: Strict provider validation and explicit separation between CI-verified `VRK_OFFLINE_SEALED_SIMULATION` and optional `LIVE_API_DISPATCH`.

Table 1: Four-layer verification architecture of The Reggae Shark Protocol (v1.1.2).

Canonical Lean 4 Module Provenance: The 24-file release archive contains 23 hash-verified files plus `MANIFEST_SHA256.txt`. Specifically, `lean/BoundaryVerification.lean` (`SHA-256: 49229b61...e417`, 9 theorems) is byte-for-byte identical to the canonical module in Physics v1.5 Appendix A (10.5281/zenodo.23534449), and `lean/ExistenceVerification.lean` (`SHA-256: eec94868...bcae`, 17 theorems) is byte-for-byte identical to the canonical module in *The Theory of Bounded Existence* v1.0.0 (10.5281/zenodo.23608244). Together with the 8 theorems in `lean/TypestateLifecycle.lean`, the package compiles **34 zero-`sorry` Lean 4 theorems**.

2 What Is and Is Not Modelled (Epistemic Claim Firewall)

Epistemic Standard (`Build. Attack. Falsify. Preserve.`): Every theorem and runtime guard in this repository is bounded by explicit domain assumptions and `scope.does_not_imply` declarations.

- **Application-Layer Typestate vs. Native Runtimes:** `lean/TypestateLifecycle.lean` proves properties of the author's formal state-transition model $\delta : \mathcal{S} \times \Sigma \to \mathcal{S}$, and `RequestSlot` / `WorkerEpochGuard` enforce those invariants in Python. They formalize a lifecycle model that rules out partial-initialization dereferencing and cross-epoch worker-slot reads within the gateway state machine; they are *not* a patch to native C/PHP interpreters or an OS ring-0 hardware memory boundary.
- **Offline Simulation vs. External Cloud APIs:** Automated CI exercises and verifies only the deterministic local pipeline (`VRK_OFFLINE_SEALED_SIMULATION`) and the local `127.0.0.1` HTTP loopback proxy. Live external cloud dispatch (`LIVE_API_DISPATCH` to Groq, Together AI, Gemini, or Hugging Face) requires external user API keys and is not claimed as verified in CI.
- **Dimensionless 4D ODE Model:** The variable $\Psi(t) \in [0,1]$ in Eq. (4) is a dimensionless mathematical self-feedback readout coordinate, not a claim of machine consciousness or sentience.

3 Layer 2 & Layer 3 Mathematical Equations

The Layer 2 governor evolves $x(t) = (\gamma(t), H_{\text{obs}}(t), \dot{I}(t), \Psi(t))$ under the smooth (C^∞) soft-wall system [3]:

$$\frac{d\gamma}{dt} = a\dot{I}\mathcal{E}_{\text{avail}}\left(1 - \frac{\gamma}{\gamma_{\max}}\right) - b\gamma, \tag{1}$$

$$\frac{dH_{\text{obs}}}{dt} = c\gamma(H_{\max} - H_{\text{obs}}) - \dot{I}_{\text{erase}}\tanh\left(\frac{H_{\text{obs}}}{\epsilon_H}\right), \tag{2}$$

$$\frac{d\dot{I}}{dt} = e\tanh\left(\frac{\dot{I}}{\epsilon_I}\right)\frac{dH_{\text{obs}}}{dt} + f\mathcal{P}_{\text{eff}}(\gamma, \Psi) - g\dot{I}, \tag{3}$$

$$\frac{d\Psi}{dt} = \kappa_{\text{meta}}R_{\text{self}}(x)(1 - \Psi) - \mu_\Psi\Psi, \tag{4}$$

where $\mathcal{P}_{\text{eff}}(\gamma, \Psi) = \mathcal{P}_0 + \lambda_{\text{sim}}r_{\text{LRSC}}\gamma + \lambda_{\text{lucid}}\Psi$ and $R_{\text{self}}(x) = \text{fb}(x)^2/(\text{fb}(x)^2 + (f\mathcal{P}_0)^2 + \sigma_0^2)$ with $\text{fb}(x) = e\tanh(\dot{I}/\epsilon_I)(dH_{\text{obs}}/dt)$.

4 Senior Red-Team Audit Board: Five Stress Resolutions

1. **Problem 1 (Continuous-to-Discrete Bridge Gap):** While Nagumo's theorem guarantees continuous-time invariance $x(t) \in \mathcal{B}_4$, an explicit discrete solver can overshoot if Δt exceeds the local stiffness limit. We resolve this by implementing `step_ode_patankar_interval` using the unconditionally positive Patankar–Euler production-destruction scheme

$$x_{k+1} = \frac{x_k + \Delta t\, P(x_k)\, x_{\max}}{1 + \Delta t\,(P(x_k) + D(x_k))}, \qquad P(x_k) \geq 0,\ D(x_k) \geq 0, \tag{5}$$

   paired with 2-ULP outward-rounded IEEE-754 interval enclosures `[interval_lo, interval_hi]`, and we machine-check in Lean 4 (`patankar_euler_step_in_box`) that $0 \leq x_k \leq x_{\max} \implies 0 \leq x_{k+1} \leq x_{\max}$ for all $\Delta t > 0$.

2. **Problem 2 ($\mathcal{P}_0 = 0$ Basin Separatrix & Capacitive Floor):** Since $R_{\text{self}}(x) \geq 0$ and $\Psi \in [0, 1]$, Theorem `psi_decay_floor_bound` in `lean/TypestateLifecycle.lean` proves $d\Psi/dt \geq -\mu_\Psi\Psi$, which by Grönwall's inequality yields the exact lower bound $\Psi(t + \Delta t) \geq \Psi(t)e^{-\mu_\Psi\Delta t}$. Runtime method `evaluate_basin_separatrix_guard` enforces both this Grönwall floor and the cycle-persistence floor $\mathcal{P}_{\text{eff}}(t) \geq \mathcal{P}_{\text{eff}}^{\text{floor}}$, raising `BasinCollapseError` fail-closed if violated.

3. **Problem 3 (LRSC $S_{11}^\perp$ Null-Space Adversarial Injection):** While the 10^{-3} LRSC threshold holds on the rank-41 odd-ring channel subspace $\text{Im}(V)$ [4], an unconstrained vector $q \in S_{11}^\perp \subset \mathbb{R}^{99}$ supported strictly on the 88 discarded indices has coordinate-sampling error $\|(I - P_{S_{11}})q\|_2/\|q\|_2 = 1$. `LRSCCompressor.compress` therefore computes the exact orthogonal energy ratio $\eta_\perp(x) = \|(I - P_{S_{11}})\tilde{x}\|_2/\|\tilde{x}\|_2$ on every call; when $\eta_\perp(x) > \tau_{\text{null}}$, it flags `null_space_anomaly_detected` to `True` and seals the full 99-mode uncompressed SHA-256 digest (`UNCOMPRESSED_FULL_RING_SEAL`).

4. **Problem 4 (Concurrency & Thread-Synchronized Lifecycle Fences):** `RequestSlot` and `WorkerEpochGuard` wrap all state transitions in a re-entrant mutex (`threading.RLock`), increment a monotonic fence counter `_fence_seq` on every transition, and provide `atomic_lifecycle_scope()` to poison and zero slots upon any mid-flight exception (verified across 16 concurrent worker threads in `test_18`).

5. **Problem 5 (Distributed Split-Brain Merkle-DAG Reconciliation):** When distributed gateway nodes process requests in different arrival orders during a partition, `merge_divergent_ledgers()` verifies both branches back to the VRK trust root and emits a deterministic merge claim with canonically sorted `parent_hashes = sorted([tip_A, tip_B])`. Theorem `merkle_dag_merge_commutative` in `lean/TypestateLifecycle.lean` proves that canonical parent-pair ordering is strictly commutative.

References

[1] P. Upadhyay, *IntentSeal Verifiable Research Kernel (VRK): Final Release v0.3.1*, Zenodo, 2026. doi:10.5281/zenodo.23018459.

[2] P. Upadhyay, *Appendix A v1.5 — Global Dissipative Completion and Soft-Wall Limit-Cycle Certification*, Zenodo, 2026. doi:10.5281/zenodo.23534449.

[3] P. Upadhyay, *The Theory of Bounded Existence: A 4D Closed-Loop Dynamical Model of Self-Modeling Thresholds, Hopf Bifurcations, and Ψ-Memory-Sustained Oscillations (v1.0.0)*, Zenodo, 2026. doi:10.5281/zenodo.23608244.

[4] P. Upadhyay, *LRSC Odd-Ring Spectral Rank Theorem (v1.2) & Exact $M = 99$ Certificate*, OSF, 2026. doi:10.17605/OSF.IO/NM5BW, doi:10.17605/OSF.IO/JAK5X.